

SPRING DOCS

Spring Boot Basics

Auto-Configuration, Starters, Dependency Injection & the Spring Ecosystem

Java Agentic AI — Knowledge Base Reference

1. What Problem Does Spring Boot Solve?

The Spring Framework (2003) pioneered Dependency Injection and Inversion of Control for Java, but configuring it (XML beans, manual server setup, dependency version matching) was notoriously heavy. Spring Boot (2014) is an opinionated layer on top of Spring that provides: **auto-configuration** (sensible defaults inferred from the classpath), **starter dependencies** (curated dependency bundles), an **embedded server** (no external Tomcat/Jetty install needed), and production-ready features (Actuator) — enabling a runnable application with minimal setup.

2. Inversion of Control & Dependency Injection

IoC means the framework — not your code — controls object creation and wiring. Dependency Injection (DI) is the specific mechanism: instead of a class creating its own dependencies with 'new', they are supplied ('injected') from outside, usually by the Spring **ApplicationContext** (the IoC container).

```
@Service
public class OrderService {
    private final PaymentService paymentService; // dependency

    @Autowired // constructor injection (preferred)
    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}

@Service
public class PaymentServiceImpl implements PaymentService { ... }
```

Injection type	How	Recommended?
Constructor injection	Via constructor parameters	Yes — enables immutability (final fields), makes required dependencies explicit
Setter injection	Via a setter method	Only for optional dependencies
Field injection	@Autowired directly on a field	Discouraged — hides dependencies, harder to unit test, can't use final fields

3. Spring Beans and the ApplicationContext

A **bean** is any object managed by the Spring IoC container. Beans are declared via stereotype annotations or explicit @Bean methods, then the container handles their full lifecycle: instantiation, dependency injection, initialization callbacks, and destruction.

```
@Component // generic stereotype
@Service // business logic layer
@Repository // data access layer (also enables exception translation)
@Controller / @RestController // web layer

@Configuration
public class AppConfig {
    @Bean
    public RestTemplate restTemplate() {
```

```

        return new RestTemplate();
    }
}

```

Scope	Meaning
singleton (default)	One shared instance per Spring container
prototype	New instance every time the bean is requested
request	One instance per HTTP request (web-aware contexts)
session	One instance per HTTP session

4. @SpringBootApplication

```

@SpringBootApplication // = @Configuration + @EnableAutoConfiguration + @ComponentScan
public class MyApp {
    public static void main(String[] args) {
        SpringApplication.run(MyApp.class, args);
    }
}

```

- **@Configuration** — marks the class as a source of bean definitions.
- **@EnableAutoConfiguration** — triggers Spring Boot's auto-configuration engine, which inspects the classpath and existing beans to configure sensible defaults (e.g. seeing spring-webmvc on the classpath auto-configures an embedded Tomcat and DispatcherServlet).
- **@ComponentScan** — scans the package (and sub-packages) of the annotated class for @Component/@Service/@Repository/@Controller beans.

5. Starters

Starters are curated dependency descriptors that pull in a coherent, version-matched set of libraries for a given concern, eliminating manual dependency-version bookkeeping.

Starter	Brings in
spring-boot-starter-web	Spring MVC, embedded Tomcat, Jackson (JSON)
spring-boot-starter-data-jpa	Spring Data JPA, Hibernate, JDBC
spring-boot-starter-security	Spring Security
spring-boot-starter-test	JUnit 5, Mockito, AssertJ, Spring Test
spring-boot-starter-actuator	Production monitoring endpoints (health, metrics, info)

6. application.properties / application.yml

```
# application.properties
server.port=8081
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.username=root
spring.jpa.hibernate.ddl-auto=update
logging.level.org.springframework=INFO
```

```
# application.yml (equivalent, YAML style)
server:
  port: 8081
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/mydb
```

```
@Value("${server.port}")
private int port;
```

```
@ConfigurationProperties(prefix = "app")
@Component
public class AppProperties {
    private String name;
    private String version;
    // getters/setters
}
```

Note: Spring Boot supports **Profiles** (*application-dev.yml*, *application-prod.yml*, activated via *spring.profiles.active*) so different environments load different configuration without code changes.

7. Actuator

spring-boot-starter-actuator exposes production-ready monitoring endpoints out of the box.

Endpoint	Purpose
/actuator/health	Application/dependency health status (DB, disk space, etc.)
/actuator/info	Arbitrary build/app metadata
/actuator/metrics	JVM, HTTP, and custom metrics (integrates with Micrometer/Prometheus)
/actuator/env	Current environment properties

8. Auto-Configuration Internals

Auto-configuration classes are conditionally applied using `@Conditional` family annotations, so they only activate when appropriate — this is why adding a dependency (like an H2 driver) can 'magically' configure a `DataSource` bean without any explicit code.

```
@Configuration
@ConditionalOnClass(DataSource.class)           // only if DataSource is on the classpath
@ConditionalOnMissingBean(DataSource.class)      // only if the user hasn't defined their
own
public class DataSourceAutoConfiguration {
```

```
@Bean  
public DataSource dataSource() { ... }  
}
```

9. Quick Interview Recap

- Spring vs Spring Boot? Spring is the core framework (IoC, AOP, MVC, etc.); Spring Boot is an opinionated layer on top providing auto-configuration, starters, and embedded servers to speed up setup and reduce boilerplate.
- Why prefer constructor injection? It supports immutable (final) dependencies, makes required dependencies explicit in the constructor signature, and allows the class to be unit tested without a Spring container by simply calling 'new'.
- What does @ConditionalOnMissingBean do? Lets auto-configuration back off gracefully if the developer has already declared their own bean of that type, so custom configuration always wins.
- Default bean scope? Singleton — one shared instance per Spring container, unless declared otherwise.